DOCUMENTATION

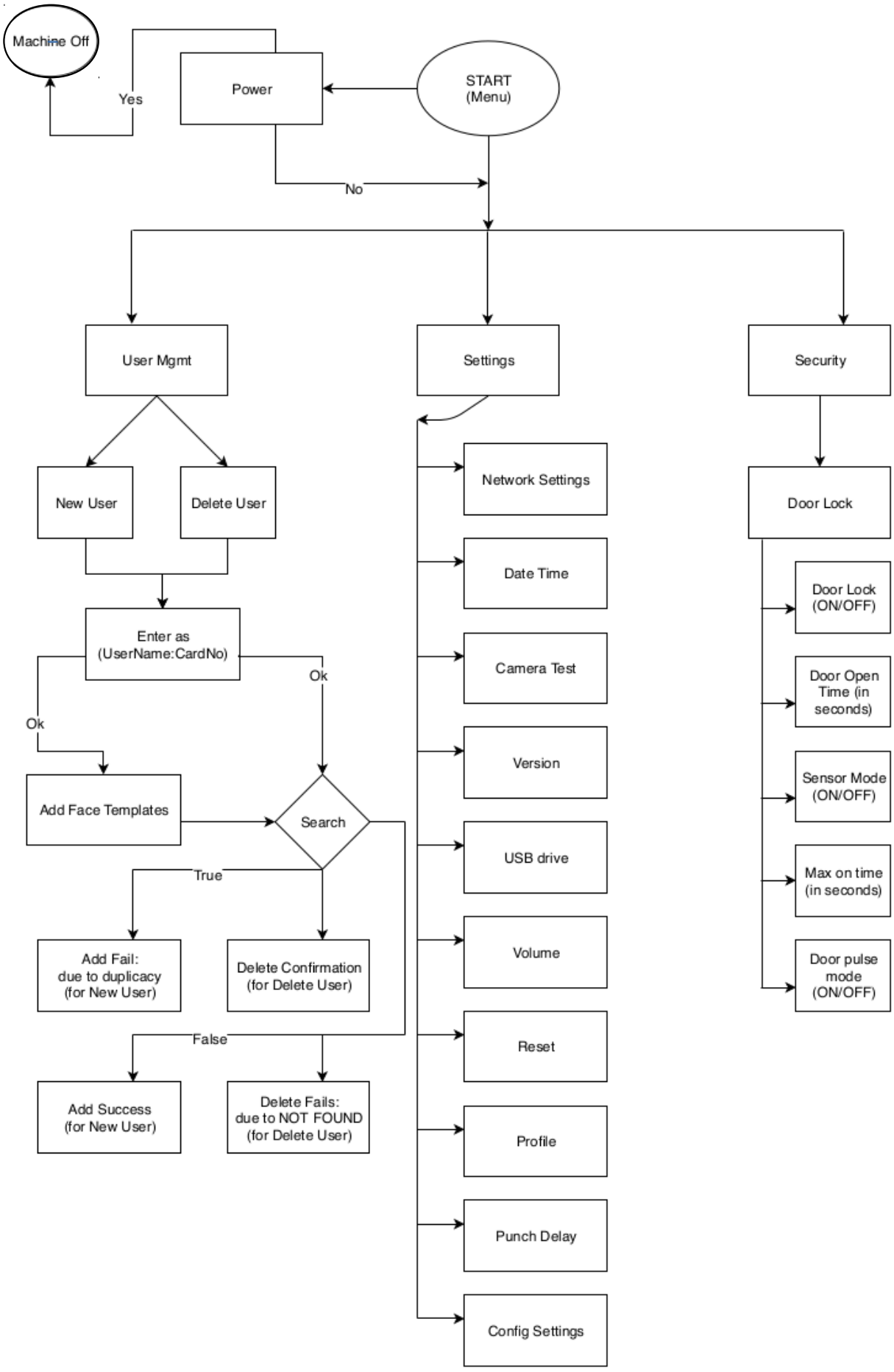
Bio Face 17

Release Date: 10 August, 2020

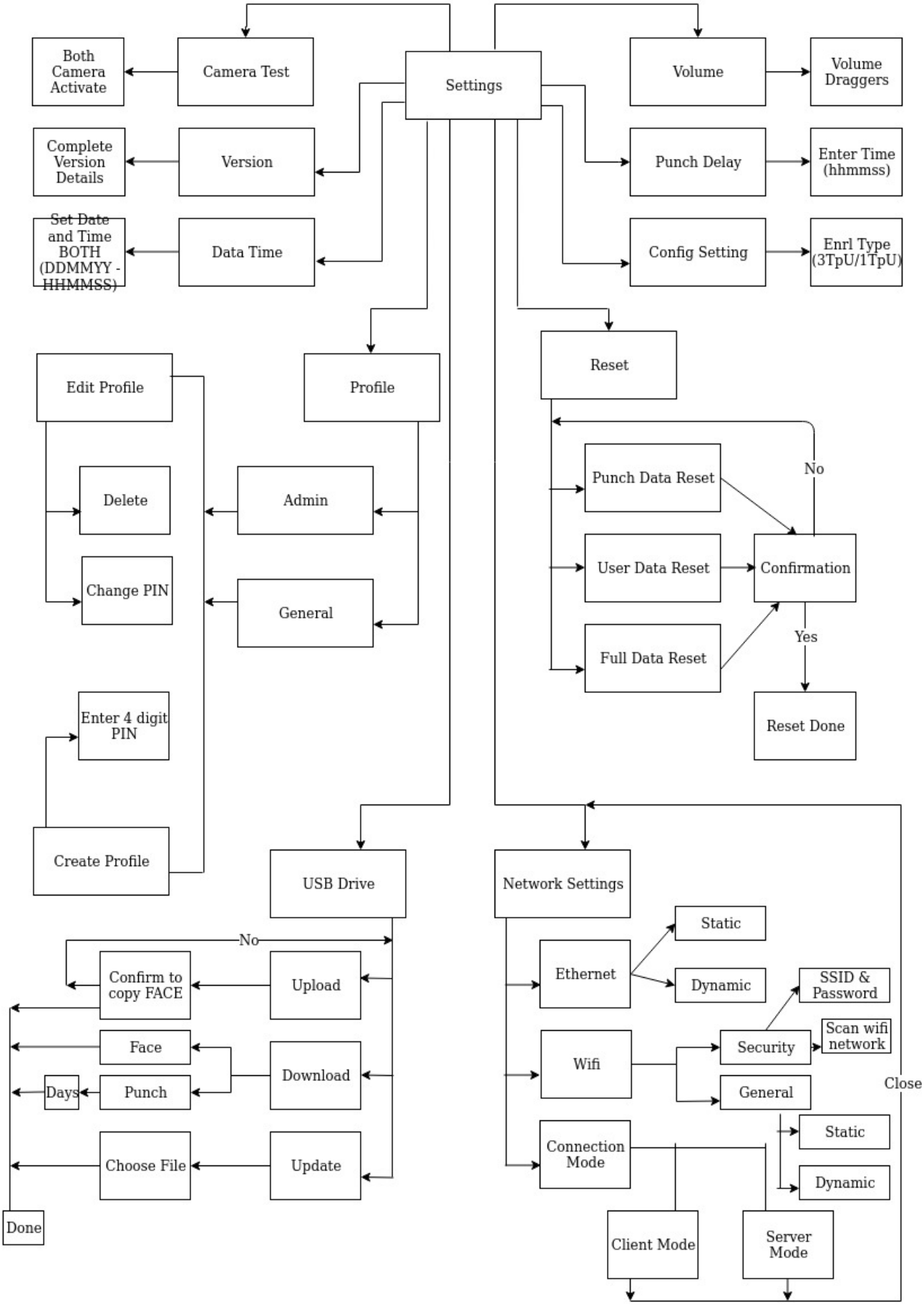
Key Points

1.	Charging:	15W	
2.	Screen Size:	240 x 320	
3.	Color Cam Size:	240 x 160	
4.	IR Cam Size:	240 x 160	
5.	Icon Size:	80 x 30	
6.	Employee Name Length:	0 to 16	
7.	Employee Card Number Length:	4 to 8	
8.	Card Number Length:	10	
9.	Face Enroll Time out:	100 sec	
10.	Card Enroll Time out:	100 sec	
11.	Profile (Admin/General) PIN Length:	4	
12.	New User Enroll Format:	NAME:CARD	(CARD = Emp Code)

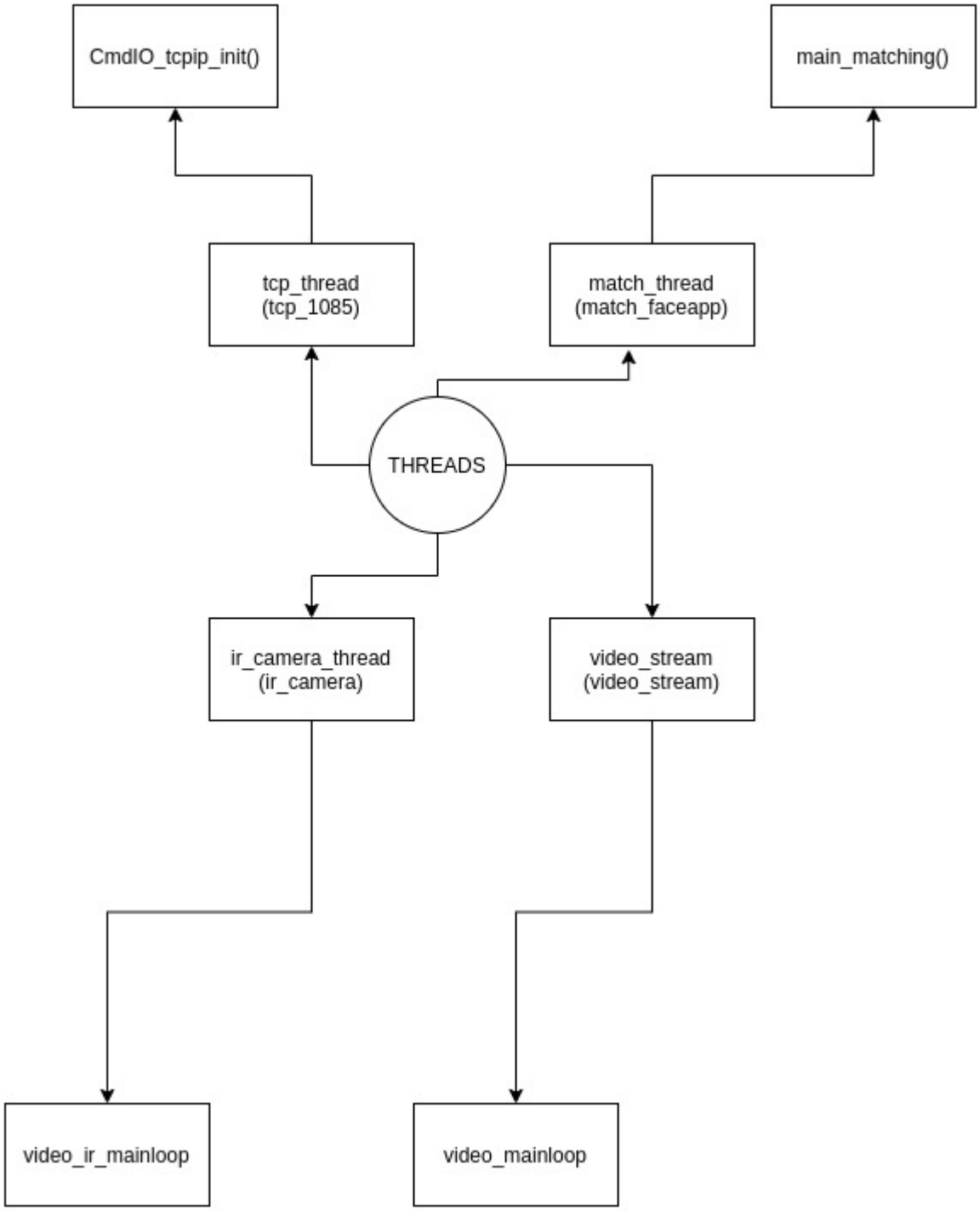
FLOW CHART


Machine Off
Yes
Power
START
(Menu)
No
User Mgmt
Settings
Security
New User
Delete User
Enter as
(UserName:CardNo)
Ok
Ok
Add Face Templates
Search
True
Add Fail:
due to duplicacy
(for New User)
Delete Confirmation
(for Delete User)
False
Add Success
(for New User)
Delete Fails:
due to NOT FOUND
(for Delete User)
Network Settings
Date Time
Camera Test
Version
USB drive
Volume
Reset
Profile
Punch Delay
Config Settings
Door Lock
Door Lock
(ON/OFF)
Door Open
Time (in
seconds)
Sensor Mode
(ON/OFF)
Max on time
(in seconds)
Door pulse
mode
(ON/OFF)

SETTINGS FLOW CHART


Settings
Camera Test
Both Camera Activate
Version
Complete Version Details
Data Time
Set Date and Time BOTH (DDMMYY - HHMMSS)
Volume
Volume Draggers
Punch Delay
Enter Time (hhmmss)
Config Setting
Enrl Type (3TpU/1TpU)
Profile
Edit Profile
Delete
Change PIN
Admin
General
Enter 4 digit PIN
Create Profile
Reset
Punch Data Reset
User Data Reset
Full Data Reset
Confirmation
No
Yes
Reset Done
USB Drive
Upload
Confirm to copy FACE
No
Download
Face
Punch
Days
Update
Choose File
Done
Network Settings
Ethernet
Static
Dynamic
Wifi
Security
SSID & Password
Scan wifi network
General
Connection Mode
Static
Dynamic
Client Mode
Server Mode
Close

THREADS FLOW CHART


CmdIO_tcpip_init()
main_matching()
tcp_thread
(tcp_1085)
match_thread
(match_faceapp)
THREADS
ir_camera_thread
(ir_camera)
video_stream
(video_stream)
video_ir_mainloop
video_mainloop

Running Threads

1. **tcp_thread**
   - This thread is responsible for making tcp connection on 1085 port.
   - Unique ID given to this thread is: “tcp_1085”.
   - Function associated with this thread: “CmdIO_io_handler”.
   - Attribute used: “PTHREAD_CREATE_DETACHED”.

2. **match_thread**
   - This thread is responsible for matching face captured from the enrolled data.
   - Unique ID given to this thread is: “match_faceapp”.
   - Function associated with this thread: “main_matching”.
   - Attribute used: “PTHREAD_CREATE_DETACHED”.

3. **ir_camera_thread**
   - This thread is reponsible for taking image input from IR camera.
   - Unique ID given to this thread is: “ir_camera”.
   - Function associated with this thread: “video_ir_mainloop”.
   - Attribute used: “PTHREAD_CREATE_DETACHED”.

4. **video_stream**
   - This thread is responsible for taking image input from colour camera.
   - Unique ID given to this thread is: “video_stream”.
   - Function associated with this thread: “video_mainloop”.
   - Attribute used: “PTHREAD_CREATE_DETACHED”.

soccamera.c

Header File(s) Used:

1. #include <linux/videodev2.h>
2. #include <sys/ioctl.h>
3. #include <sys/mman.h>
4. #include <fcntl.h>
5. #include <unistd.h>
6. #include <string.h>
7. #include <errno.h>
8. #include <stdlib.h>
9. #include <stdio.h>
10. #include <pthread.h>
11. #include <signal.h>

12. #include "drv_display_sun4i.h"
13. #include "dragonboard_inc.h"
14. #include "Face_Define.h"

Functions Used:

1. monitor_color_camera_thread
2. process_image
3. draw_square
4. video_thread_kill_wait
5. video_thread_start
6. disp_init
7. disp_start
8. disp_stop
9. disp_quit
10. disp_set_addr
11. read_frame
12. video_mainloop
13. sigint_handler
14. main
15. ir_camera_thread_start
16. read_ir_frame
17. video_ir_mainloop

Function Details

1. **main()**

main function first check command line arguments passed and start lynxFaceApp accordingly, for example if we write following:

lynxFaceApp -m -8

The above command will start application with width and height set on index 8 of array
(a) array_width and
(b) array_height
i.e. 720x720.

Further, memory blocks are created for following buffers:
1. irdisp_buff
2. irgray_buff
3. irprev_buff
4. nv12_buff
5. mcamera_buff
using malloc function.

Next, attributes for pthreads is set as detachable. Then start creating threads with unique names. Using pthread_create and pthread_setname_np().

THREAD	SET NAME	METHOD CALLED
tcp_thread	tcp_1085	CmdIO_io_handler
match_thread	match_faceapp	main_matching
ir_camera_thread	ir_camera	video_ir_mainloop
video_stream	video_stream	video_mainloop

Next, main_gtk() is called.

Next, threads are joined with main thread. Using pthread_join().

At last, clear buffer for
1. nv12_buff
2. mcamera_buff
using free() and return -1.

2. **video_ir_mainloop()**
   - open ir camera named: “/dev/camera-ir” in O_RDWR and O_NONBLOCK mode.

3. **video_mainloop()**
   - open color camera named: “/dev/camera-usb” in O_RDWR and O_NONBLOCK mode.

4. **monitor_color_camera_thread()**
   - call “pthread_kill” for “video_stream”
   - check return value is greater than 0. If yes, call video_thread_start()

5. **video_thread_start()**
   - call “pthread_create” for “video_stream” and calling method “video_mainloop”
   - check return value is greater than 0. If yes, return 1
   - set thread name as: “video_stream”
   - return 0

6. **process_image()**
   - parameters:
     - const void *p
     - int size
   - open “image.raw” in “w+” mode.
   - Write “p” data in “image.raw”.

7. **draw_square()**
   - parameters:
     - uint8_t *pixelin
     - uint32_t w
     - uint32_t h
     - uint32_t x
     - uint32_t y
     - uint32_t size
   - write in pixel using memset()

8. **video_thread_kill_wait()**
   - call “pthread_join” for “video_stream”

9. **disp_start()**
   - set args[0] = 0 and args[2] = layer
   - call ioctl for “disp” with request “DISP_CMD_VIDEO_START”
   - return value returned by above ioctl

10. **disp_stop()**
   - set args[0] = 0 and args[2] = layer
   - call ioctl for “disp” with request “DISP_CMD_VIDEO_STOP”
   - return value returned by above ioctl

11. **disp_quit()**
   - set args[0] = 0 and args[2] = layer
   - call ioctl for “disp” with request “DISP_CMD_VIDEO_CLOSE”
   - call ioctl for “disp” with request “DISP_CMD_VIDEO_RELEASE”
   - return 0

12. **disp_init()**
    - parameters:
        - int width
        - int height
    - open “/dev/disp” in “O_RDWR”
    - store and check return value == -1 in “disp”. If yes,
        - show db_error with error number
        - return -1
    - set args[0]
    - set screen_width as return value of ioctl for “disp” with request “DISP_CMD_SCN_GET_WIDTH”
    - set screen_height as return value of ioctl for “disp” with request “DISP_CMD_SCN_GET_HEIGHT”
    - set args[0] = 0 and args[2] = DISP_LAYER_WORK_MODE_SCALER
    - set layer as return value of ioctl for “disp” with request “DISP_CMD_SCN_GET_REQUEST”
    - check if layer == 0. If yes,
        - show db_error with error number
        - return -1
    - set layer_para using memset
    - set all attributes of layer_para
    - return 0

13. **disp_set_addr()**
    - parameters:
        - int width
        - int height
        - unsigned int *addr
    - set fb using memset
    - set attributes of fb
    - check csi_format and set addr attribute of fb
    - set args
    - call ioctl for “disp” with request “DISP_CMD_VIDEO_SET_FB”
    - return value returned by above ioctl

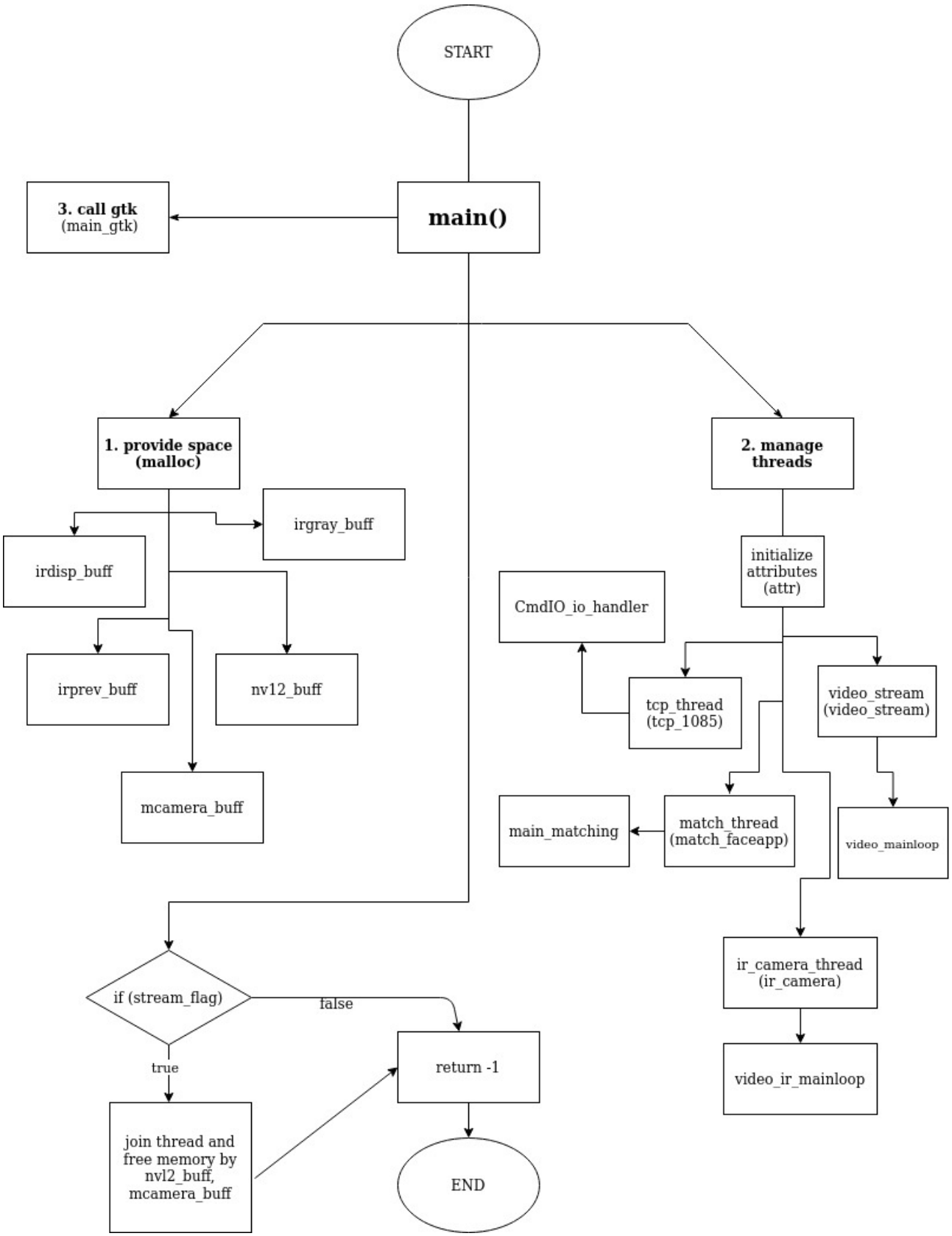
14. **sigint_handler()**
    - parameter:
        - int sig
    - call pthread_join for “match_thread”
    - check stream_flag == 2. If yes,
        - join “video_stream” thread
        - check if fd is greater than 0, then call ioctl for fd with “VIDIOC_STREAMOFF”
        - unmapbuffer if mapped
        - check if data in buffers, then free buffers using free
        - check if fd is greater than 0, then close fd
    - free memory by irgray_buff, irdisp_buff, irprev_buff using free()
    - call signal with SIG_DFL
    - call kill with getpid()
    - call exit

15. **ir_camera_thread_start()**
    - check and store return value of pthread_create for “ir_camera_thread” for method “video_ir_mainloop” and attribute “attr” in code. If it return value greater than zero then,
        - return 1
    - set pthread name of ir_camera_thread as “ir_camera”
    - return 0

16. **read_ir_frame()**
    - set buf using memset()
    - set attributes for buf
    - check frame.status == false
        - check status == -1. If yes, return 0
        - check frameCount > 5. If yes,
            - memcpy ir_buffers[buf.index].start into frame.buffer
            - set frameCount = 6
        - check sq.status == 0xaa. If yes, set sq.status = 0
    - set frame.status = TRUE

17. **read_frame()**
    - set buf using memset()
    - set attributes for buf
    - check return value of ioctl for “fd” with “VIDIOC_DQBUF”.
        - If yes,
            - check mframe.status == FALSE. If yes,
                - memcpy buffers[buf.index].start into mframe.buffer
                - set mframe.status = TRUE
        - If not,
            - return 0
    - call ioctl for “fd” with “VIDIOC_QBUF”
    - increment out_buf by 1
    - return 1

SOCCAMERA.C
FLOW CHART


START
main()
3. call gtk
(main_gtk)
1. provide space
(malloc)
irgray_buff
irdisp_buff
irprev_buff
nv12_buff
mcamera_buff
2. manage
threads
initialize
attributes
(attr)
CmdIO_io_handler
tcp_thread
(tcp_1085)
video_stream
(video_stream)
main_matching
match_thread
(match_faceapp)
video_mainloop
ir_camera_thread
(ir_camera)
video_ir_mainloop
if (stream_flag)
false
true
return -1
join thread and
free memory by
nvl2_buff,
mcamera_buff
END

gtkGUI.c

Header File(s) Used:

1. #include <gtk/gtk.h>
2. #include <gtk/gtkx.h>
3. #include <stdio.h>
4. #include <stdlib.h>
5. #include <glib.h>
6. #include <arpa/inet.h>
7. #include <string.h>
8. #include <sqlite3.h>
9. #include <time.h>
10. #include <sys/stat.h>
11. #include <dirent.h>

12. #include "Face_Define.h"
13. #include "gpio_lib.h"
14. #include "ll_port.h"
15. #include "lib_Crypto.h"

Functions Used:

1. resize_bw_image_to_rgb
2. crop_in_center_rgb_image
3. draw_square_colour_camera
4. draw_fix_square_colour_camera
5. cryptoRead
6. cryptoMem
7. draw_cross
8. isValidIp4
9. edit_mode
10. empty_string
11. update
12. generate_log
13. set_button_label_default
14. hide_unused_icon
15. cancel_task
16. ID_delete
17. check_user
18. enroll_ID
19. set_password
20. enter_password
21. menu_function
22. destroy_popup
23. device_version
24. firmware_update
25. run_update
26. update_window
27. update_confirmation

28. power_shutdown
29. ethernet_dynamic
30. wireless_dynamic
31. wireless_info
32. eth_info
33. ethernet_window
34. wlan_window
35. network_setting
36. camera_open
37. camera_close
38. save_date_time
39. set_date_time
40. power_saving_out
41. capture_image
42. open_keyboard
43. close_keyboard
44. switch_state_update
45. save_lock_settings
46. door_lock_mode
47. set_volume
48. get_volume
49. face_mode_change
50. main_gtk

Function Details

1. **resize_bw_image_to_rgb**
   - parameters:
     - const unsigned char *input_pixels
     - int input_w
     - int input_h
     - unsigned char *output_pixels
     - int output_w
     - int output_h
   - convert black and white image to rgb color image

2. **crop_in_center_rgb_image**
   - parameters:
     - const unsigned char *input_pixels
     - int input_w
     - int input_h
     - unsigned char *output_pixels
     - int output_w
     - int output_h

3. **draw_square_colour_camera**
   - parameters:
     - uint8_t *pixelin
     - uint32_t w
     - uint32_t h
     - uint32_t x
     - uint32_t y
     - uint32_t size
   - draw square when machine found face in front
   - square color will be white

4. **draw_fix_square_colour_camera**
   - parameters
     - uint8_t *pixelin
     - uint32_t w
     - uint32_t h
     - uint32_t x
     - uint32_t y
     - uint32_t size
   - draw square permanently on screen
   - square color will be green

5. **cryptoRead**
   - check if LockActive is 1.
   - If yes,
     - set senstivity of “doorTime” and “SensorSwitch” as TRUE
     - check “SensorActive” is 0. If yes,

        - set state of “OnOffSwitch” as TRUE
        - set senstivity of “doorTime” and “SensorSwitch” as GTK_STATE_FLAG_INSENSITIVE
        - check “SensorActive” is 0. If yes,
            - set gtk state flag of “sensorTime” as GTK_STATE_FLAG_INSENSITIVE
            - set gtk state of “SensorSwitch” as FLASE
        - else check “SensorActive” is 1. If yes,
            - set senstivity of “sensorTime” as TRUE
            - set gtk state of “SensorSwitch” as TRUE
    - else check if “LockActive” is 1. If yes,
        - set state of “OnOffSwitch” as FALSE
        - set senstivity of “doorTime” as GTK_STATE_FLAG_INSENSITIVE
        - check “SensorActive” is 0. If yes,
            - set gtk state flag of “sensorTime” as GTK_STATE_FLAG_INSENSITIVE
            - set gtk state of “SensorSwitch” as FLASE
        - else check “SensorActive” is 1. If yes,
            - set senstivity of “sensorTime” as TRUE
            - set gtk state of “SensorSwitch” as TRUE
- If not,
    - set gtk state of “OnOffSwitch” as FALSE
    - set gtk state of “pulseMode” as TRUE
    - check “SensorActive” is 0. If yes,
        - set gtk state flag of “sensorTime” as GTK_STATE_FLAG_INSENSITIVE
        - set gtk state of “SensorSwitch” as FLASE
    - else check “SensorActive” is 1. If yes,
        - set senstivity of “sensorTime” as TRUE
        - set gtk state of “SensorSwitch” as TRUE
    - set gtk state flag of “doorTime” and “OnOffSwitch” as GTK_STATE_FLAG_INSENSITIVE
- return 0

6. **cryptoMem**
    - parameters:
        - char * data
        - int len

7. **draw_cross**
    - parameters:
        - uint8_t *pixelin
        - uint32_t w
        - uint32_t h
        - uint32_t size

8. **isValidIp4**
    - parameter:
        - const char * ip
    - use “inet_pton” function to verify IP address
    - returns return value of above function

9. **edit_mode**
    - parameter:
        - GtkWidget *widget

- delete buffer
- set textbox editable
- set widget label text as “OK”
- set focus on textbox

10. **empty_string**
    - parameters:
        - char array[]
        - int size
    - set null value to received array

11. **generate_log**
    - parameter:
        - char buff[]
    - check if mode is not “MATCH” and log_flag is 0 and switchFlag is 0. Then,
        - xx
    - else if, mode is “MATCH” and switchFlag is 0
        - xx
    - else if, switchFlag is 1
        - xx
    - return

12. **set_button_label_default**
    - set label text of “Button” as “New\nUser”
    - set label text of “Del_Button” as “Delete\nUser”
    - set label text of “Set” as “Password”
    - redraw Button, Del_ Button, Set and textbox widgets

13. **hide_unused_icon**
    - parameter:
        - GtkWidget * ActiveIcon
    - hide following widgets:
        - power_off
        - Button
        - Del_Button
        - version
        - date_time
        - volume
        - camera
        - network
        - firmware
        - Set
        - doorLock
        - ActiveIcon

14. **check_user**
    - parameters:
        - char * userID
    - try open sqlite3 database using “sqlite3_open” and store return value in rc
    - check rc is not equal to “SQLITE_OK”. If yes,
        - close database
        - return

- run select query for “userID”
- fetch table using “sqlite3_get_table” and store rows in “rows”
- if rows is not zero
  - free table using “sqlite3_free_table”
  - close database using “sqlite3_close”
  - return 1
- free table using “sqlite3_free_table”
- close database using “sqlite3_close”
- return 0

15. **firmware_update**
    - creates new window (GTK_WINDOW_POPUP)
    - size: 240x320
    - add combo box populated by “ItemList”

16. **update_window**
    - create new window (GTK_WINDOW_TOPLEVEL)
    - size: 240x320
    - run command “/bin/mount |grep -c media” and store in fp and check if fp is null
      - If yes,
        - insert “Failed To Run Command” in combo box “ItemList”
      - If not,
        - fetch path return by the command and check if null.
          - If yes,
            - insert “NO USB DEVICE FOUND” in combo box “ItemList”
          - If not,
            - run command “/bin/mount |grep media”
            - look for any .tar file
            - show “No Patch file found” if not found

17. **get_volume**
    - fetch current volume of machine and store in val.
    - devide val by 100.00 and store in “value”
    - set this “value” to scale button “volume”

18. **main_gtk**
    main_gtk function declare all GtkWidget and place them.

“g_timeout_add” FUNCTIONS

1. **update**
   This function check flags and hide/unhide screen icons and other widgets.

“clicked” EVENT FUNCTIONS

1. **menu_function**
   This function fetch password from “PSWD_PATH” and check if is null or not.
   If yes,
   - set mode = 0
   - delete buffer
   - redraw textbox
   - hide all main screen widget

   If not,
   - unparent textbox
   - remove “fixed” from “window”
   - delete buffer
   - put textbox into “fixed3”
   - redraw textbox
   - hide all main screen widget

2. **enter_password**
   This function check label text of button “Enter” is “Enter\nPassword” or not.
   If yes,
   - set purpose of textbox as “GTK_INPUT_PURPOSE_PASSWORD”
   - toggle keyboard
   - call edit_mode()
   - set mode = 0
   - append password to “PSWRD_PATH”

   If not,
   - hide textbox
   - get buffer length
   - check length
   - if length == 4
     - get buffer
     - check if buffer data == password strored
       - set password null
       - unparent textbox
       - remove fixed3 from window
       - put textbox into fixed2
       - delete buffer
       - add fixed2 into window
       - redraw fixed2
       - hide textbox
       - hide all main screen widget
     - else if buffer data == “7397”
       - set mode = PASSWORD
       - set second_elapsed = 0
       - set password = null
       - remove pwd_config.txt
       - delete buffer
       - show “Password Removed” and set to buffer also

        - show textbox
    - else
        - delete buffer
        - show “Wrong Try again”
        - set dialogFlag = 1
        - set “Enter” button label as "Enter\nPassword"
        - set mode = PASSWORD
        - set password null
        - second_elapsed = 0
        - redraw “Enter”
- else
    - check length == 0
        - if yes, show “Password should not be blank”
        - if not, show “Wrong Try again”
    - set dialogFlag = 1
    - set password null
    - set mode = PASSWORD
    - set second_elapsed = 0

3. **set_password**

   This function check label text of button “Set” is “OK” or not.
   If yes,
   - call edit_mode for “Set”
   - show textbox
   - check label text of button “Del_Button” is “OK” or not.
     - If yes, set label text as “Delete\nUser”
   - check label text of button “Button” is “OK” or not.
     - If yes, set label text as “New\nUser”

   If not,
   - hide Set and textbox
   - get length of buffer
   - check if length == 4
     - if yes,
       - open PSWRD_PATH file in write mode
       - get buffer data
       - show “Password Set”
       - set dialogFlag = 1
     - if not,
       - check if length == 0, show “Password should not be blank”
       - else, show “Only 4 Digit Alphanumeric”
       - set dialogFlag = 1

4. **cancel_task**
   - Check if label text of button “Button” is “New\nUser” and “Del_Button” is “Delete\ nUser” and “Set” is “Password” and mode is not ENROLL. Then,
     - set loop_wait = 0
     - set text.status = 3
     - set IR_PWR: HIGH
     - set mode = MATCH
     - set frame_timeout = time()
     - set gtk state flag for textbox as GTK_STATE_FLAG_INSENSITIVE
     - check if enroll_flag is greater than 0. If yes,

        - remove main_image from fixed2
        - add main_image in fixed
        - set enroll_flag = 0
    - set frame.status FALSE
    - delete buffer
    - set “Enter” button label as “Enter\nPassword”
    - unparent textbox
    - put textbox in fixed
    - redraw textbox, fixed2, fixed
    - remove fixed2 from window
    - hide main screen widgets
- else if mode == ENROLL
    - set quality = 0
    - set mode = EXTRA
    - set loop_wait = 1
    - FaceCount = 1
    - delete buffer
    - set state flag of textbox as “GTK_STATE_FLAG_NORMAL”
    - remove main_image from fixed2
    - put main_image in fixed
    - unparent textbox
    - put textbox in fixed2
    - hide main screen widgets
    - check enroll_flag == 2. If yes,
        - set IR_PWR: LOW
        - set stream_flag = 0
        - set enroll_flag = 0
    - set IR_PWR: LOW
    - set enroll_flag = 0
- else
    - hide FaceMode
    - hide FaceCount = 1
    - hide textbox and main screen widgets
    - set IR_PWR: LOW

5. **ID_delete**

   Check if label text of “Del_Button” is “OK” or not.
   If yes,
   - check if label text of “Button” as “OK”. If yes,
     - set button label text as “New\nUser”
   - check if label text of “Set” as “OK”. If yes,
     - set button label text as “Password”

   If not,
   - get length of the buffer
   - check buffer length is greater than 1 and less than 9.
     - If yes,
       - get buffer data
       - set loop_wait = 0
       - set mode = DELETE
       - set text.status = 4
       - show “Please Wait...”
     - If not,

        - check if data length less than 1,
            - if yes, show “Please input card number”
            - if not, show “Card Number shoule be not more than 8 digit in length”
        - set dialogFlag = 1
    - put null char at name[length]
    - set “Del_Button” label text as “Delete\nUser”
    - redraw Del_Button and textbox

6. **enroll_ID**
   Check label text of “Button” is “OK” or not.
   If yes,
    - show textbox and FaceMode
    - set gtk state flag of FaceMode as GTK_STATE_FLAG_NORMAL
    - check label text of “Del_button” is “OK”. If yes, set label text as “Delete\nUser”
    - check label text of “Set” is “OK”. If yes, set label text as “Password”

   If not,
    - hide textbox, Button and FaceMode
    - get length of buffer
    - if length is greater than zero
        - get buffer data
        - set card = strchr(data, ‘:’)
        - check card is not null and length of card is greater than 4. Then,
            - If yes,
                - check input as
                    - no special character
                    - card name lenght greater than 0 and less than equal to 16
                    - employee card number length greater than equal to 4 and less than equal to 8
            - If not,
                - check if card is null.
                    - If yes,
                        - show “Format is NAME:CARD”
                    - If not,
                        - show “Card Number Length\nShould be in between 4 to 8”
                - set dialogFlag = 1
                - set char_count = 0
                - set spl_char = 0
    - else
        - show “Please input card number”
        - set dialogFlag = 1
    - set “Button” lable text as “New\nUser”
    - redraw Button and textbox

7. **update_confirmation**
   This function will run command throught popen and store return value in fp then check fp is it null or not.
   If yes,
    - set flags = GTK_DIALOG_DESTROY_WITH_PARENT
    - show dialog with message as “Failed to run Command”

   If not,
    - store data into path
    - check if path is null

- if yes,
  - set flags = GTK_DIALOG_DESTROY_WITH_PARENT
  - show dialog message as “No usb device found”
- if not,
  - call update_window()

8. **network_setting**
   - Open “/etc/network/interfaces” in read mode and store return value in fp. If fp is not null then,
     - fetch data in file_buff
     - check and set buff_copy when buff_data has sub string “wlan”. If yes,
       - set data when buff_copy has sub string “inet”.
         - Check if data is null then return
       - check w_config.type not equal to “dhcp”. If yes,
         - set data when buff_copy has sub string “address” and check data not equal to null, then
           - assign data += 8
           - append w_config.ip_address and data
         - set data when buff_copy has sub string “netmask” and check data not equal to null, then
           - assign data += 8
           - append w_config.netmask and data
         - set data when buff_copy has sub string “gateway” and check data not equal to null, then
           - assign data += 8
           - append w_config.gateway and data
         - set data when buff_copy has sub string “nameservers” and check data not equal to null, then
           - assign data += 12
           - append w_config.dns and data
       - set data when buff_copy has sub string “ssid” and check data not equal to null, then
         - assign data += 5
         - append w_config.ssid and data
       - set data when buff_copy has sub string “psk” and check data not equal to null, then
         - assign data += 4
         - append w_config.pskey and data
       - set data when buff_copy has sub string “ether” and check data not equal to null, then
         - assign data += 6
         - append e_config.hwaddress and data
     - check and set buff_copy when buff_data has sub string “eth”. If yes,
       - set data when buff_copy has sub string “inet” and check data not equal to null, then
         - assign data += 5
         - append e_config.type and data
       - check e_config.type is “dhcp”.
         - If yes,
           - set data when buff_copy has sub string “ether” and check data not equal to null, then
             - assign data += 6

        - append e_config.hwaddress and data
    - If not,
        - set data when buff_copy has sub string “address” and check data not equal to null, then
            - assign data += 8
            - append e_config.ip_address and data
        - set data when buff_copy has sub string “netmask” and check data not equal to null, then
            - assign data += 8
            - append e_config.netmask and data
        - set data when buff_copy has sub string “gateway” and check data not equal to null, then
            - assign data += 8
            - append e_config.gateway and data
        - set data when buff_copy has sub string “nameservers” and check data not equal to null, then
            - assign data += 12
            - append e_config.dns and data
        - set data when buff_copy has sub string “address” and check data not equal to null, then
            - assign data += 6
            - append e_config.hwaddress and data
- This creates a new window (GTK_WINDOW_TOPLEVEL)

9. **device_version**
    - This creates a new window (GTK_WINDOW_POPUP)
    - size: 240x320
    - create a scrolled window (gtk_scrolled_window_new)
    - collect and display information like:
        - ROM Date
        - SDK Version
        - Version
        - MMC Memory
        - AUTO DELETION DAYS
        - MAC address
        - Total Memory
        - Free Memory

10. **power_shutdown**
    - Check label text of button “Del_Button” is “OK”.
        - If yes, set label text as “Delete\nUser”
    - Check label text of button “Button” is “OK”.
        - If yes, set label text as “New\nUser”
    - Check label text of button “Set” is “OK”.
        - If yes, set label text as “Password”
    - Show dialog with “Yes” and “No” button and message as “Do you really want to turn off device?”
    - Check user response is “GTK_RESPONSE_YES”.
        - If yes, run system call “poweroff”
        - If not, call gtk_widget_destroy

11. **camera_open**
    - hide keyboard
    - set IR_PWR: HIGH
    - remove main_image from fixed
    - add main_image into color_cam
    - call gtk_widget_show_all for data widget

12. **set_date_time**
    - This creates a new window (GTK_WINDOW_TOPLEVEL)
    - run and store the result of command “date +%d%m%y”

13. **door_lock_mode**
    - This creates a new window (GTK_WINDOW_TOPLEVEL).

14. **face_mode_change**
    Check is toggle button “FaceMode” is active or not.
    - If yes, set FaceCount = 3
    - If not, set FaceCount = 1

15. **destroy_popup**
    - destroy gtk widget “user_data” using “gtk_widget_destroy”
    - toggle keyboard

16. **run_update**
    - set flags = GTK_DIALOG_MODAL | GTK_DIALOG_DESTROY_WITH_PARENT
    - show dialog box with “Yes” - “No” button and message “Do you want to update?”
    - check user response.
        - If user response is “GTK_RESPONSE_YES”
            - call “firmware_update”
            - destroy dialog box and break
        - else
            - destroy dialog box and break

17. **eth_info**
    - check eth_count == 0
        - If yes,
            - take input from user and check is valid by calling isValidIp4() of followings:
                - IP Address
                - Netmask
                - DNS
                - Gateway
            - if is not valid show “Wrong Entry” and return
            - check and store return value of isValidIp4 for dns in result is zero
                - If yes,
                    - open file “/tmp/interfaces” in “w” mode
                    - write message in file and close file
                    - copy “/tmp/interfaces” to “/etc/network/”
                - If not,
                    - open file “/tmp/interfaces” in “w” mode
                    - write message in file and close file
                    - copy “/tmp/interfaces” to “/etc/network/”
        - If not,

        - check and store return value of isValidIp4 for dns in result is zero
            - If yes,
                - open file “/tmp/interfaces” in “w” mode
                - write message in file and close file
                - copy “/tmp/interfaces” to “/etc/network/”
            - If not,
                - open file “/tmp/interfaces” in “w” mode
                - write message in file and close file
                - copy “/tmp/interfaces” to “/etc/network/”
- show dialog box with “Yes” - “No” button and message “Reboot Now to make changes effective?”
- check user response
    - If user response is “GTK_RESPONSE_YES”, call system method “reboot”
    - else, destroy dialog box

18. **ethernet_dynamic**
    Check if eth_count is zero.
    - If yes,
        - set gtk state flag of entry, entry2, entry3, entry4 as “GTK_STATE_FLAG_INSENSITIVE”
        - set eth_count = 1
    - If not,
        - set senstivity of entry, entry2, entry3, entry4 as “TRUE”
        - set eth_count = 0

19. **wireless_info**
    - check tab_name is not “security”
        - If yes,
            - check wlan_count == 0
                - If yes,
                    - check and store return value of isValidIp4 for dns in result is zero
                        - IP Address
                        - Netmask
                        - DNS
                        - Gateway
                    - if is not valid show “Wrong Entry” and return
                - If not,
                    - append w_config.type with “dhcp”
        - If not,
            - fetch ssid data
            - check length of ssid is less than 7. If yes,
                - show “Minimum 8 words”
                - return
            - fetch pskey data
            - check length of pskey is less than 7. If yes,
                - show “Minimum 8 words”
                - return
    - check wlan_count == 0
        - open file “/tmp/interfaces” in “w” mode
        - write message and close
        - copy “/tmp/interfaces” into “/etc/network/”
    - else if check wlan_count == 1

    - open file “/tmp/interfaces” in “w” mode
    - write message and close
    - copy “/tmp/interfaces” into “/etc/network/”
- show dialog box with “Yes” - “No” button and message “Saved Successfully\n\nReboot Now?”
- check user response,
    - if response is “GTK_RESPONSE_YES”
        - run system call “reboot”
        - break
    - else
        - destroy dialog box
- return

20. **wireless_dynamic**

    Check if wlan_count is zero
    - If yes,
        - set gtk state flag of entry5, entry6, entry7, entry8 as “GTK_STATE_FLAG_INSENSITIVE”
        - set eth_count = 1
    - If not,
        - set senstivity of entry5, entry6, entry7, entry8 as “TRUE”
        - set eth_count = 0

21. **ethernet_window**
    - create new gtk window (GTK_WINDOW_TOPLEVEL)
    - size: 240x320
    - create and display widgets
    - check e_config.type == “dhcp”
        - If yes,
            - set eth_count = 1
            - set state flag of entry, entry2, entry3, entry4 as “GTK_STATE_FLAG_INSENSITIVE”
        - If not,
            - set text of entry, entry2, entry3, entry4 as e_config.ip_address, e_config.netmask, e_config.dns , e_config.gateway

22. **wlan_window**
    - create new gtk window (GTK_WINDOW_TOPLEVEL)
    - size: 240x320
    - create and display widgets
    - check w_config.type == “dhcp”
        - If yes,
            - set wlan_count = 1
            - set state flag of entry5, entry6, entry7, entry8 as “GTK_STATE_FLAG_INSENSITIVE”
        - If not,
            - set text of entry5, entry6, entry7, entry8 as w_config.ip_address, w_config.netmask, w_config.dns , w_config.gateway

23. **save_date_time**
    - fetch data and time from entry buffer “date_buff” and “time_buff” respectively.
    - count numericals occurs in date and time
    - check length of date less than 6 or length of time less than 6 or count greater than 0
        - delete text in “date_buff” and “time_buff”
        - show “Wrong Entry” as placeholder in “date_entry” and “time_entry”
        - set count = 0
        - return
    - set count = 0
    - set date and time in correct format (appending colons)
    - run date command with appropiate options save in “cmd” and run system call
    - compare cmd buff
        - If same,
            - delete text in “date_buff” and “time_buff”
            - show “Date Changed” and “Time Changed” as placeholder in “date_entry” and “time_entry” respectively
        - If not,
            - delete text in “date_buff” and “time_buff”
            - show “Not Set” as placeholder in “date_entry” and “time_entry”

24. **save_lock_settings**
    - check if gtk switch state of “OnOffSwitch”, “pulseMode” and “SensorSwitch” is TRUE or FALSE
        - If state is true then value of “LockActive”, “pulseActive” and “SensorActive” is set to be 1
        - If state is false then value will set to be 0
        - append 0/1 whatever the value set above to info+strlen(info)
        - set LOCK_RLY: LOW
    - set flags = GTK_DIALOG_MODAL | GTK_DIALOG_DESTROY_WITH_PARENT
    - show dialog boc with OK button and message as “Lock Settings Saved”

“value-changed” EVENT FUNCTIONS

1. **set_volume**
   This will fetch requested volume in double type variable “value” initialized as 0.0. Then it is miltipled by 100 upto 2 decimal places and append to the command: “amixer set Master <requested_volume>”. Finally run system call to set requested volume.

“notify::state” EVENT FUNCTIONS

1. **switch_state_update**
   This will set senstivity and state flag of OnOffSwitch, pulseMode and sensorTime as TRUE as per the conditions on state of these widgets either TRUE or FALSE.

“destroy” EVENT FUNCTIONS

1. **gtk_main_quit**
   This is the gtk function. This makes the innermost invocation of the main loop return when it regains control.

“button_press_event” EVENT FUNCTIONS

1. **camera_close**
   - remove main_image from color_cam
   - add main_image in fixed
   - set IR_PWR: LOW
   - hide data

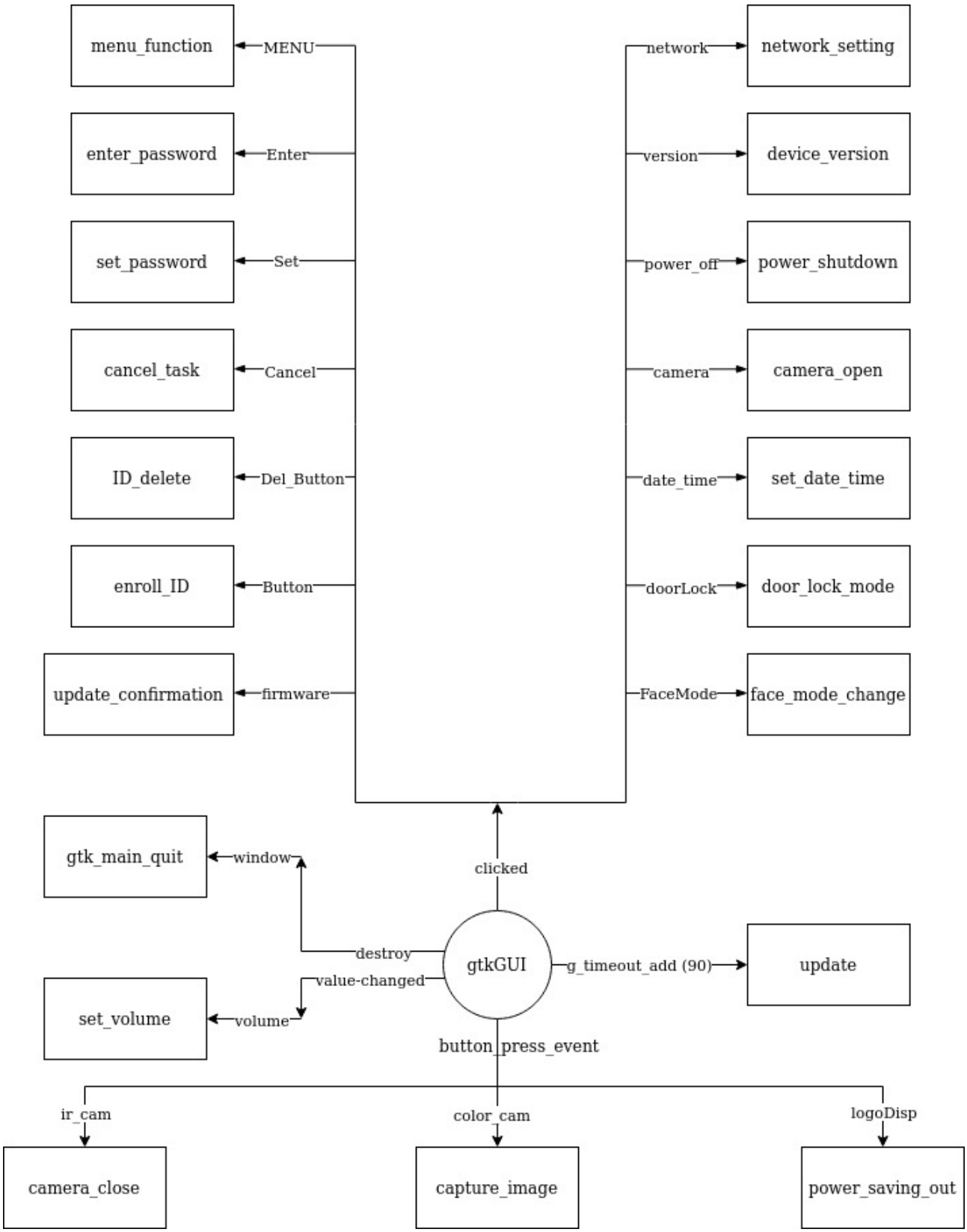
2. **capture_image**
   - set loop_wait = 0
   - call strftime
   - set mode = CAPTURE
   - show “Image Captured”
   - set dialogFlag = 1
   - call camera_close

3. **power_saving_out**
   This function check stream_flag == 0. If yes,
   - show all widget in window widget
   - hide all main screen widgets
   - calculate sleep time and if less than 2 sec: sleep for 1 more sec
   - hide logoTime and logoDate widget
   - set stream_flag = 1
   - set IR_PWR: HIGH
   - start ir camera thread
   - store frame_timeout as time(NULL)

4. **open_keyboard**
   - toggle keyboard to close (in case)
   - make system call to open

5. **close_keyboard**
   - toggle keyboard to close

gtkGUI.C
FLOW CHART


menu_function
MENU
enter_password
Enter
set_password
Set
cancel_task
Cancel
ID_delete
Del_Button
enroll_ID
Button
update_confirmation
firmware
network
network_setting
version
device_version
power_off
power_shutdown
camera
camera_open
date_time
set_date_time
doorLock
door_lock_mode
FaceMode
face_mode_change
gtk_main_quit
window
clicked
destroy
gtkGUI
g_timeout_add (90)
update
value-changed
set_volume
volume
button_press_event
ir_cam
color_cam
logoDisp
camera_close
capture_image
power_saving_out

Critical Variables

1. struct square sq	:	check and set “status” of square
2. int framePsec	:	set frame per second
3. int saver_flag	:	check machine is in power saving mode
4. int TempCount	:	To count face number in enroll mode , right now upto 3 face side used to enroll
5. char sdk_version[1025]	:	store machine info about threshold and enrol
6. int loop_wait	:	to stop while loop of match process when enter in menu mode
7. int dialogFlag	:	show dialog box when true
8. int image_flag	:	
9. struct sframe frame	:	
10. struct sframe mframe	:	
11. int quality	:	quality of face captured from camera
12. int red	:	font color of text
13. int green	:	font color of text
14. int blue	:	font color of text
15. int FaceCount	:	set configure settings
16. unsigned char *irgray_buff	:	store ir camera data
17. unsigned char *irprev_buff	:	store ir camera data
18. unsigned char *irdisp_buff	:	store ir camera data
19. char mode	:	To select diffrent modes i.e, 'm' = Matching , 'e' = Enroll, & 'd' = Delete
20. char task[8]	:	Task name to write in log , contain one at a time among Enroll,Delete and Match
21. int stream_flag	:	check is match_thread create or not
22. struct TextData text	:	check status and name
23. int LockActive	:	if lock settings are active
24. SensorActivepulseActive	:	
25. int durration	:	time for how long door can remain open after relay lock set to open
26. int alarmTime	:	maximum door open time after which sensor will start alarming the message of door open
27. char face_buff[1024*12]	:	Buffer used to carry extracted user data from database in hex format
28. unsigned char disp_buff[240*320*3]	:	
29. int avgValue	:	
30. int backendFlag	:	
31. time_t frame_timeout	:	to calculate idle time out when no face/object found on camera frame
32. float frameNum	:	
33. int log_flag	:	When enroll or delete done successfully this flag set to 0 else remain 1
34. unsigned int read_flag	:	
35. unsigned int temp_size	:	Size of template data while performing enroll and extraction process from back end cmd
36. volatile char pixel_buff[]	:	set 0xff for half screen size

37. volatile char pix_buff[]	:	set 0x00 for half screen size
38. char FunctionStatus[100]	:	
39. volatile char userId[32]	:	
40. volatile char name[32]	:	User Id enter in gui for delete or enroll process
41. volatile char time_stamp[]	:	To save image with time and date added in the image file name
42. char rgb_buff[]	:	

DATA LOCATION

1. Log File : /var/log/FaceApp.log
2. Config File : .config/lxsession/Linaro/autostart
3. Network File : /etc/network/interfaces
4. Icons : /root/Documents/Icons/